Looping Statements in Python: A Comprehensive Guide

Loops are fundamental programming constructs that allow you to execute a block of code repeatedly. Python provides two primary types of loops: `for` and `while`.

1. The `for` Loop

The `for` loop is used to iterate over a sequence (such as a list, tuple, dictionary, set, or string).

Basic Syntax

```
for item in sequence:
    # Code block to execute
```

Example: Iterating over a List

```
fruits = ["apple", "banana", "cherry"]
for fruit in fruits:
    print(f"I like {fruit}")
```

Using `range()`

The `range()` function generates a sequence of numbers, which is commonly used with `for` loops.

```
# Iterates from 0 to 4
for i in range(5):
    print(i)
```

2. The `while` Loop

The `while` loop executes a block of code as long as a specified condition is `True`.

Basic Syntax

```
while condition:
    # Code block to execute
```

Example: Counting down

```
count = 5
while count > 0:
    print(f"Countdown: {count}")
    count -= 1  # Important: update the condition variable to avoid infinite loop
print("Blast off!")
```

3. Nested Loops

A nested loop is a loop inside another loop. The "inner loop" will be executed one full time for each iteration of the "outer loop".

Example: Multiplication Table (Partial)

```
for i in range(1, 4):       # Outer loop
    for j in range(1, 4):   # Inner loop
        print(f"{i} * {j} = {i*j}")
    print("--- End of inner loop ---")
```

4. Loop Control Statements

Control statements change the execution of a loop from its normal sequence.

A. `break` Statement

Used to exit the loop prematurely when a certain condition is met.

```
for num in range(1, 10):
    if num == 5:
        break  # Stops the loop entirely when num is 5
    print(num)
```

B. `continue` Statement

Skips the current iteration and moves to the next one.

```
for num in range(1, 6):
    if num == 3:
        continue  # Skips the print for 3
```

```
    print(num)
```

C. `pass` Statement

The `pass` statement is a null operation; it does nothing. It is used as a placeholder when a statement is syntactically required but you don't want any command or code to execute.

```
for x in [0, 1, 2]:
    if x == 1:
        pass  # Placeholder for future code
    print(f"Current value: {x}")
```

Summary Table

Statement	Description
for	Iterates over a sequence of items.
while	Repeats as long as a condition is True.
break	Terminate the loop statement and transfers execution to the statement immediately following the loop.
continue	Causes the loop to skip the remainder of its body and immediately retest its condition prior to reiterating.
pass	Used when a statement is required syntactically but you do not want any code to execute.